

Objective

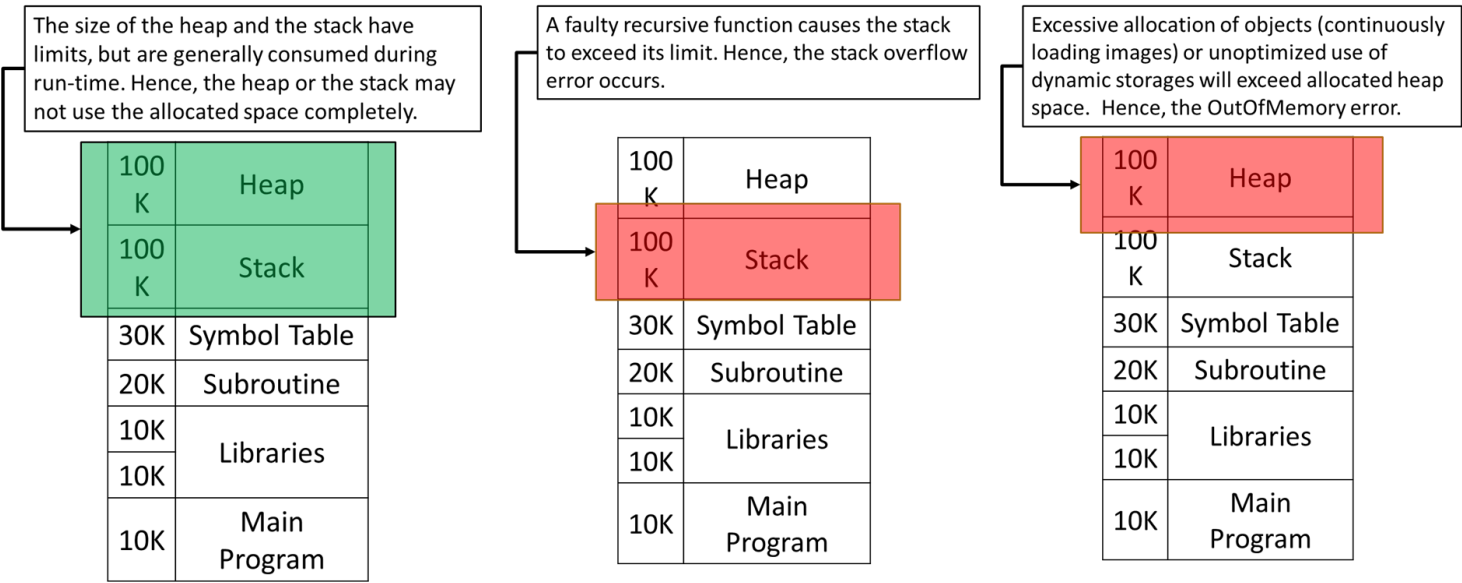
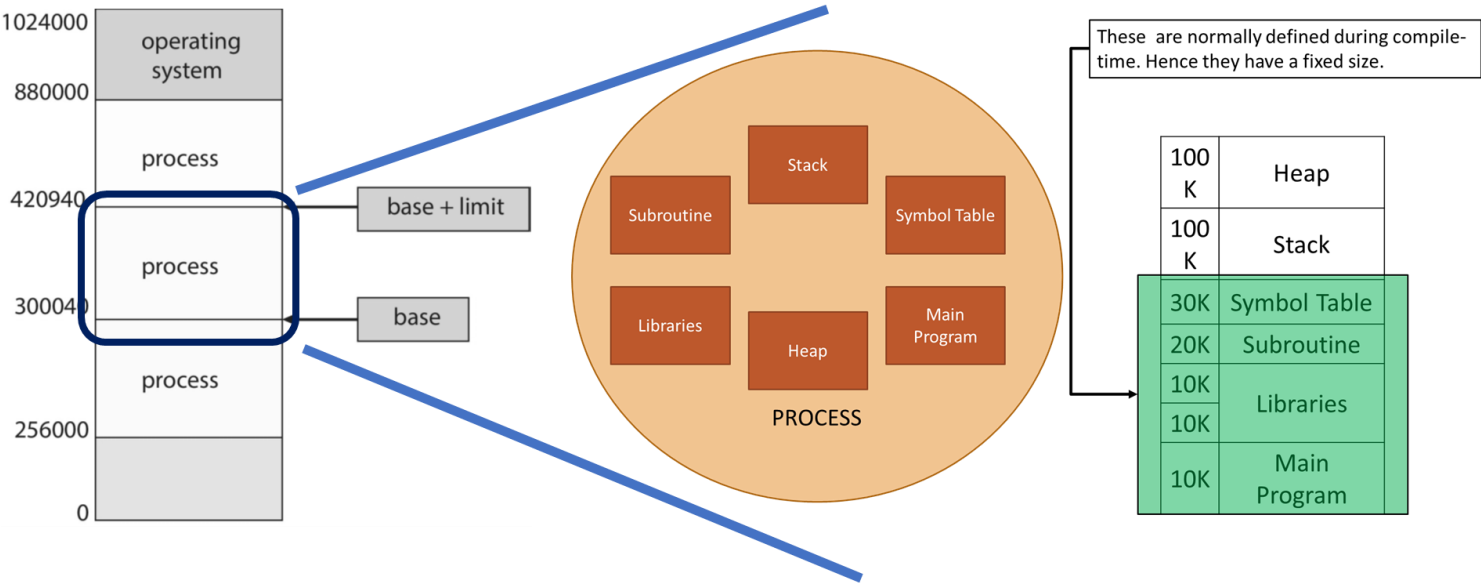
- How OS manages its main memory for multiple processes.
- How to emulate memory management for a multi-tasking OS.

Overview

- There are several kinds of memory management models. We will only discuss:
 - Contiguous memory or flat memory model
 - Paging
 - Demand paging

Basic Principles

- Most OS pre-allocate memory for each process, depending on its relative size and computational demand. The pre-allocated memory can be written/freed when executing instructions → malloc(), dealloc(), pointers, etc.
 - Each process have a separate memory space.
 - Start address and end address present in process info.
- OS also has a share of its memory space!
- Processes cannot execute without their resources loaded into memory.



An Illustration of Flat Memory Allocation

Dynamic Memory Allocation Problem

- At the very start of the OS program, memory are divided into N partitions, with pre-defined sizes.

10K	
10K	
10K	
10K	
10K	
10K	
10K	
10K	
--	OS



Dynamic Memory Allocation Problem

- Before the process executes in the CPU, the OS allocates a hole big enough for the process.

10K	
10K	
10K	
10K	
10K	
10K	
10K	
10K	Process 1
--	OS



Dynamic Memory Allocation Problem

- Processes that require larger memory should need more holes.
- Consider P2 that requires 20K memory

10K	
10K	
10K	
10K	
10K	
10K	Process 2
10K	Process 1
--	OS



Dynamic Memory Allocation Problem

- Processes sometimes require memory that is not divisible by partition size
- Consider P3 that requires 14K memory.
- P3 still needs to consume an additional 10K hole!

10K	
10K	
10K	
10K	Process 3
10K	Process 2
10K	Process 1
--	OS



Dynamic Memory Allocation Problem

- Unused memory in hole results in **internal fragmentation**.

P3 needs additional 10K hole. But 4K only used. Hence, internal fragmentation of 6K.

10K	
10K	
10K	
10K	Process 3
10K	Process 2
10K	Process 1
--	OS



Dynamic Memory Allocation Problem

- QUESTION: What will the OS do if a process arrives, but the memory is full? (+1)

Process 5 ❌

10K	Process 4
10K	Process 3
10K	Process 2
10K	Process 1
--	OS



Dynamic Memory Allocation Problem

- Incoming processes to be scheduled in CPU may be delayed if memory space is full.
- OS may do the following:
 - Wait until a process finishes execution, to free up memory space
 - Perform swapping. A process in the waiting queue will have to release its memory and transfer it back to a **backing store**.

Process 5 ❌

10K	Process 4
10K	Process 3
10K	Process 2
10K	Process 1
--	OS



Dynamic Memory Allocation Problem

- If swapping is enabled, the swapped process, when re-scheduled back to the CPU, reloads its resources back to memory from the backing store.
- Both options have tradeoff costs!
 - Waiting until a process finishes – lower CPU throughput.
 - Swapping – moving back the process to memory incurs additional CPU cost.

10K	Process 4
10K	Process 3
10K	Process 2
10K	Process 1
--	OS



Dynamic Memory Allocation Problem

- When a process finishes execution, it can immediately free memory for a future process.

10K	Process 4
10K	
10K	Process 3
10K	Process 2
10K	
--	OS



Dynamic Memory Allocation Problem

- Suppose a new process arrives that demands 30K memory.
- We have enough memory space! But at different partitions/holes. What can the OS do?

Process 5

10K	Process 4
10K	
20K	
10K	Process 2
10K	
--	OS



Dynamic Memory Allocation Problem

- The OS cannot allocate P5 in memory! Enough memory space but fragmented!
- Partitions that are not **continuous** are called **external fragmentation**.
- In this example, external fragmentation is 30K. Meaning 30K space may be left unused because it has been divided.

10K	Process 4
10K	
10K	
20K	
10K	Process 2
10K	
10K	
--	OS

Dynamic Memory Allocation Problem

- QUESTION: Assume all processes have finished execution. What can we do to our memory space to lessen fragmentation? (+1)

30K	
20K	
20K	
10K	
--	OS

Dynamic Memory Allocation Problem

- OS may choose to do the following:
 - Revert back to its original partitioning scheme.
 - Maintain new partitions as is.
 - Continuous partitions may be merged, to avoid internal fragmentation.

30K	
20K	
20K	
10K	
--	OS

Dynamic Memory Allocation Problem

- Continuous partitions may be merged, to avoid internal fragmentation.
- P6 arrives that requires 32K memory.

30K	
20K	
20K	
10K	
--	OS

Dynamic Memory Allocation Problem

- Continuous partitions may be merged, to avoid internal fragmentation.
- P6 arrives that requires 32K memory.

48K	
32K	Process 6
--	OS

Swapping and the Backing Store

The diagram shows the interaction between the operating system, user space, and main memory on one side, and the backing store on the other.
1. 'swap out': A process (P1) is moved from the user space/main memory to the backing store.
2. 'swap in': A process (P2) is moved from the backing store back to the user space/main memory.

General steps:

- Continuously allocate processes in the user space.
- If there are no free user space, select a process that could be put to a backing store. To put in backing store, save necessary information about the process. E.g. PID, program counter, certain allocations/variable declarations performed.
- Free up the block of the selected process
- Put the new process in the recently freed up block.

Handwritten notes:

- ↓ min = fragmentation
- ↑ CPU utilization
- X processes actively executing in CPU
- 1. backing store
- 2. loading it again
- core 1
- context-switching

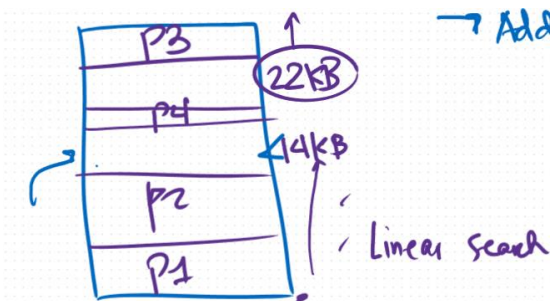
- Swapping** is a memory management technique used by operating systems to temporarily move a process or a portion of a process from main memory (RAM) to secondary storage (usually a disk) and vice versa. The secondary storage is the **backing store**.
- The primary goal of swapping is to free up space in the main memory for active processes. When a process is swapped out, its contents are moved to the secondary storage, allowing other processes to utilize the freed-up memory.
- Process:** When a process is swapped out, all of its contents are transferred to a reserved space on the disk (swap space). The operating system maintains a swapping policy to determine when to swap processes in and out based on priority, demand, or other criteria.

- **Performance Impact:** While swapping can prevent memory exhaustion and allow more processes to run, excessive swapping can lead to performance degradation due to increased I/O operations between main memory and secondary storage.

The flat memory model follows a combination of **first-fit** and **quick-fit** approaches. Here are the primary fit approaches:

1. **First Fit:**

- **Description:** In the first-fit approach, the operating system allocates the first available block of memory that is large enough to accommodate the process's size.
- **Advantages:** Simple and easy to implement. Quick allocation of memory.
- **Disadvantages:** May result in fragmentation, both internal and external, over time.



2. **Best Fit:**

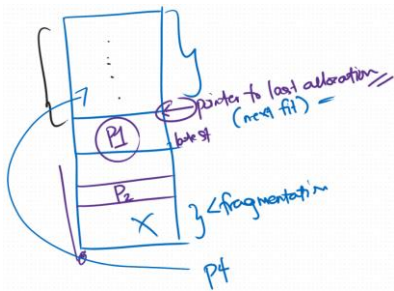
- **Description:** The best-fit approach allocates the smallest available block of memory that is large enough to fit the process.
- **Advantages:** Minimizes wasted memory by selecting the closest match in size. Reduces external fragmentation.
- **Disadvantages:** May lead to fragmentation and may require searching through the entire list of available blocks.

3. **Worst Fit:**

- **Description:** The worst-fit approach allocates the largest available block of memory. It aims to leave the largest holes for future allocations.
- **Advantages:** Can lead to larger available blocks for subsequent allocations.
- **Disadvantages:** May result in more wasted memory due to fragmentation. Like best fit, searching for the largest block can be time-consuming.

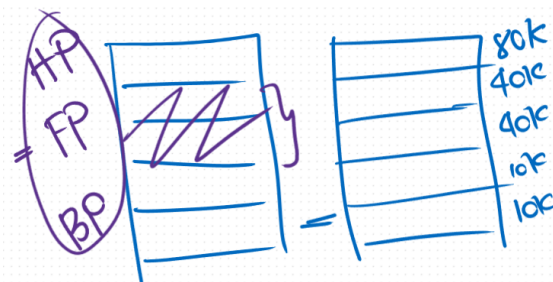
4. **Next Fit:**

- **Description:** Similar to the first-fit approach, but it starts searching for available memory from the last allocation point. It continues searching until a suitable block is found.
- **Advantages:** Reduces fragmentation compared to first fit. Simplifies implementation.
- **Disadvantages:** Still susceptible to fragmentation.



5. **Quick Fit:**

- **Description:** In quick fit, the memory is divided into predefined block sizes, and each size has its own list of available blocks. Allocation involves finding the appropriate size class and selecting the first available block.
- **Advantages:** Quick allocation, especially for commonly used block sizes.
- **Disadvantages:** May lead to fragmentation within size classes.



Each approach aims to reduce but cannot eliminate fragmentation.

Emulating a first-fit flat memory model

The necessary condition for a process to be selected for CPU execution is that an available memory block is available. If it is not available, we can either:

- Select a random process to be put into a backing store
- Put the candidate process back into the ready queue and wait for another chance to use the memory.

First, consider an OS design with **no backing store support** and use a **first-come-first-serve** scheduler.

Flat memory allocator example

```
108 //flat memory allocator with FCFS
109 #include <iostream>
110 #include <vector>
111 #include <unordered_map>
112
113 // Process class representing a process with name, id, and memory required
114 class Process {
115 public:
116     std::string name;
117     int id;
118     size_t memoryRequired;
119
120     Process(const std::string& name, int id, size_t memoryRequired)
121         : name(name), id(id), memoryRequired(memoryRequired) {}
122 };
123
124 // IMemoryAllocator interface
125 class IMemoryAllocator {
126 public:
127     virtual void* allocate(size_t size) = 0;
128     virtual void deallocate(void* ptr) = 0;
129     virtual std::string visualizeMemory() = 0;
130 };
131
```

The flat memory allocator is implemented as:

```
132 // FlatMemoryAllocator class implementing IMemoryAllocator
133 class FlatMemoryAllocator : public IMemoryAllocator {
134 public:
135     FlatMemoryAllocator(size_t maximumSize) : maximumSize(maximumSize), allocatedSize(0) {
136         memory.reserve(maximumSize);
137         initializeMemory();
138     }
139
140     ~FlatMemoryAllocator() {
141         memory.clear();
142     }
143
144     void* allocate(size_t size) override {
145         // Find the first available block that can accommodate the process
146         for (size_t i = 0; i < maximumSize - size + 1; ++i) {
147             if (!allocationMap[i] && canAllocateAt(i, size)) {
148                 allocateAt(i, size);
149                 return &memory[i];
150             }
151         }
152
153         // No available block found, return nullptr
154         return nullptr;
155     }
156
157     void deallocate(void* ptr) override {
158         // Find the index of the memory block to deallocate
159         size_t index = static_cast<char*>(ptr) - &memory[0];
160         if (allocationMap[index]) {
161             deallocateAt(index);
162         }
163     }
164
165     std::string visualizeMemory() override {
166         return std::string(memory.begin(), memory.end());
167     }
168 }
169
```

```

169 private:
170     size_t maximumSize;
171     size_t allocatedSize;
172     std::vector<char> memory;
173     std::unordered_map<size_t, bool> allocationMap;
174
175     void initializeMemory() {
176         std::fill(memory.begin(), memory.end(), '.'); // '.' represents unallocated memory
177         std::fill(allocationMap.begin(), allocationMap.end(), false);
178     }
179
180     bool canAllocateAt(size_t index, size_t size) const {
181         // Check if the memory block is large enough
182         return (index + size <= maximumSize);
183     }
184
185     void allocateAt(size_t index, size_t size) {
186         // Mark the memory block as allocated
187         std::fill(allocationMap.begin() + index, allocationMap.begin() + index + size, true);
188         allocatedSize += size;
189     }
190
191     void deallocateAt(size_t index) {
192         // Mark the memory block as deallocated
193         allocationMap[index] = false;
194     }
195 };

```

```

197 // Simple first-come-first-serve scheduler
198 class FCFS_Scheduler {
199 public:
200     FCFS_Scheduler(IMemoryAllocator& memoryAllocator) : memoryAllocator(memoryAllocator) {}
201
202     void addProcess(const Process& process) {
203         readyQueue.push_back(process);
204     }
205
206     void executeProcesses() {
207         for (const auto& process : readyQueue) {
208             void* memory = memoryAllocator.allocate(process.memoryRequired);
209
210             if (memory != nullptr) {
211                 std::cout << "Allocated memory for process " << process.name << " (ID: " << process.id << ")\n";
212                 std::cout << "Memory state: " << memoryAllocator.visualizeMemory() << "\n";
213                 memoryAllocator.deallocate(memory);
214             } else {
215                 std::cout << "Insufficient memory for process " << process.name << " (ID: " << process.id << ")\n";
216             }
217
218             //<additional instructions for executing the process in CPU is done here>
219         }
220
221         readyQueue.clear();
222     }
223
224 private:
225     IMemoryAllocator& memoryAllocator;
226     std::vector<Process> readyQueue;
227 };
228
229 int main() {
230     const size_t maximumMemorySize = 50;
231     FlatMemoryAllocator memoryAllocator(maximumMemorySize);
232     FCFS_Scheduler scheduler(memoryAllocator);
233
234     // Add processes to the scheduler
235     scheduler.addProcess(Process("Process_A", 1, 10));
236     scheduler.addProcess(Process("Process_B", 2, 5));
237     scheduler.addProcess(Process("Process_C", 3, 8));
238
239     // Execute processes
240     scheduler.executeProcesses();
241
242     return 0;
243 }

```

Concept of placement of new operator

- To effectively manage memory and have more control over which memory blocks are allocated, we use “placement new”
- Placement new is a variation new operator in C++.
 - Normal new operator does two things : (1) Allocates memory (2) Constructs an object in allocated memory.
 - Placement new allows us to separate above two things. In placement new, we can pass a preallocated memory and construct an object in the passed memory.

An example. Allocate 10 uint8_t items in a stack-based order:

```
109 void MemoryTestRun::testAllocation_uint8()
110 {
111     std::cout << "--TEST ALLOCATION. UINT8--" << std::endl;
112     const int MAX_SIZE = 10;
113
114     // Allocate a block of memory to store MAX_SIZE * sizeof(uint8_t) bytes
115     uint8_t* data = new uint8_t[MAX_SIZE * sizeof(uint8_t)];
116
117     // Pointer to keep track of the next available position (offset in bytes)
118     uint8_t* top = data;
119
120     // Push 10 values (treated as uint8_t) onto the stack using placement new
121     for (int i = 0; i < 10; ++i) {
122         new (top) uint8_t(200); // Placement new to construct uint8_t at top
123         top++;
124     }
125
126     // Access and print the elements (stack order - LIFO)
127     while (top != data) {
128         top--;
129         std::cout << "Value: " << static_cast<int>(*top) << ", Address: " << reinterpret_cast<int>(top) << std::endl;
130     }
131
132     std::cout << std::endl;
133
134     // Deallocate the memory block (no placement delete needed)
135     delete[] data;
136 }
```

Another example. Allocate 10 int32_t items in a stack-based order:

```
138 void MemoryTestRun::testAllocation_int32()
139 {
140     std::cout << "--TEST ALLOCATION. UINT32--" << std::endl;
141
142     //ALLOCATION FOR SIGNED INT
143     const int MAX_SIZE = 10;
144
145     // Allocate a block of memory to store MAX_SIZE * sizeof(uint8_t) bytes
146     int32_t* data = new int32_t[MAX_SIZE * sizeof(int32_t)];
147
148     // Pointer to keep track of the next available position (offset in bytes)
149     int32_t* top = data;
150
151     // Push values onto the stack using placement new
152     for (int i = 0; i < 10; ++i) {
153         new (top) int32_t(15235521); // Placement new to construct uint8_t at top
154         top++;
155     }
156
157     // Access and print the elements (stack order - LIFO)
158     while (top != data) {
159         top--;
160         std::cout << "Value: " << static_cast<int>(*top) << ", Address: " << reinterpret_cast<int>(top) << std::endl;
161     }
162
163     std::cout << std::endl;
164
165     // Deallocate the memory block (no placement delete needed)
166     delete[] data;
167 }
168 }
```

Below shows the sample run of these functions:

```
--TEST ALLOCATION. UINT8--
Value: 200, Address: 000002A2CBC27D49
Value: 200, Address: 000002A2CBC27D48
Value: 200, Address: 000002A2CBC27D47
Value: 200, Address: 000002A2CBC27D46
Value: 200, Address: 000002A2CBC27D45
Value: 200, Address: 000002A2CBC27D44
Value: 200, Address: 000002A2CBC27D43
Value: 200, Address: 000002A2CBC27D42
Value: 200, Address: 000002A2CBC27D41
Value: 200, Address: 000002A2CBC27D40

--TEST ALLOCATION. UINT32--
Value: 15235521, Address: 000002A2CB8CEE44
Value: 15235521, Address: 000002A2CB8CEE40
Value: 15235521, Address: 000002A2CB8CEE3C
Value: 15235521, Address: 000002A2CB8CEE38
Value: 15235521, Address: 000002A2CB8CEE34
Value: 15235521, Address: 000002A2CB8CEE30
Value: 15235521, Address: 000002A2CB8CEE2C
Value: 15235521, Address: 000002A2CB8CEE28
Value: 15235521, Address: 000002A2CB8CEE24
Value: 15235521, Address: 000002A2CB8CEE20
```

Explanation of the uint8_t and int32_t placement new allocation:

Allocation:

- Allocates a block of memory to store 10 times the size of a uint8_t using new uint8_t[MAX_SIZE * sizeof(uint8_t)].

Stack Implementation:

- Uses a uint8_t* pointer named top to keep track of the next available position, mimicking a stack.

Pushing Elements (10 times):

- Iterates 10 times.
- Inside the loop:
 - Uses new (top) uint8_t(200) to construct a uint8_t with value 200 at the memory location pointed to by top (placement new).
 - Increments top to point to the next available position.

Printing Elements (LIFO order):

- Loops until top reaches the beginning of the allocated memory.
- Inside the loop:
 - Decrements top to point to the last element pushed.
 - Prints the value of the element at *top after casting it to int for easier visualization (since uint8_t is typically printed as an unsigned char).
 - Prints the memory address of the element using reinterpret_cast<void*>(top).

Deallocation:

- Deallocates the entire memory block using delete[] data.

There may be cases where we may need to **reuse** the memory allocated. This is where the “placement new” operator is needed, as there is no need to allocate new memory.

Below is an example of our allocation using 10 int32_t items, which are then replaced by new items.

```
170 void MemoryTestRun::testReAllocation_int32()
171 {
172     std::cout << "--TEST ALLOCATION WITH OVERWRITING. INT32--" << std::endl;
173
174     //ALLOCATION FOR SIGNED INT
175     const int MAX_SIZE = 10;
176
177     // Allocate a block of memory to store MAX_SIZE * sizeof(int32) bytes
178     int32_t* data = new int32_t[MAX_SIZE * sizeof(int32_t)];
179
180     // Pointer to keep track of the next available position (offset in bytes)
181     int32_t* top = data;
182
183     // Push values onto the stack using placement new
184     for (int i = 0; i < 10; ++i) {
185         new (top) int32_t(15235521); // Placement new to construct int32 at top
186         top++;
187     }
188
189     // Access and print the elements (stack order - LIFO)
190     while (top != data) {
191         top--;
192         std::cout << "Value: " << static_cast<int>(*top) << ", Address: " << reinterpret_cast<int>(top) << std::endl;
193     }
194     std::cout << std::endl;
195     std::cout << "Re-allocating/overwriting data!" << std::endl;
196
197     //reallocate new data
198     //Push values onto the stack using placement new
199     for (int i = 0; i < 10; ++i) {
200         new (top) int32_t(-100); // Placement new to construct int32 at top
201         top++;
202     }
203
204     // Access and print the elements (stack order - LIFO)
205     while (top != data) {
206         top--;
207         std::cout << "Value: " << static_cast<int>(*top) << ", Address: " << reinterpret_cast<int>(top) << std::endl;
208     }
209
210     std::cout << std::endl;
211
212     // Deallocate the memory block (no placement delete needed)
213     delete[] data;
214 }
215 }
```

Explanation:

- Allocation for Signed Integers (int32_t):
 - Allocates a block of memory to store 10 times the size of an int32_t using new int32_t[MAX_SIZE * sizeof(int32_t)].
 - Creates a pointer top of type int32_t* to keep track of the next available position.
- Pushing Initial Values (10 times):
 - Iterates 10 times.
 - Inside the loop:
 - Uses new (top) int32_t(15235521) to construct an int32_t with value 15235521 at the memory location pointed to by top (placement new).
 - Increments top to point to the next available position.
- Printing Initial Elements (LIFO order):
 - Loops until top reaches the beginning of the allocated memory.
 - Inside the loop:
 - Decrements top to point to the last element pushed.
 - Prints the value of the element at *top after casting it to int for easier visualization.
 - Prints the memory address of the element using reinterpret_cast<int>(top).
- Re-allocating/Overwriting Data:
 - Prints a message indicating re-allocation and overwriting.
- Overwriting Existing Values (10 times):
 - Iterates 10 times again.
 - Inside the loop:
 - Uses new (top) int32_t(-100) to construct an int32_t with value -100 at the same memory locations used previously (overwriting existing values).
 - Increments top to point to the next available position (although it's not actually used in this case).
- Printing Overwritten Elements (LIFO order):
 - Loops until the top reaches the beginning of the allocated memory.
 - Inside the loop:
 - Decrements top to point to the last overwritten element.
 - Prints the value (now -100) and memory address as before.

7. Deallocation:
- Deallocates the entire memory block using delete[] data. This frees the memory even though the data was overwritten.

Purpose of placement new with OS memory management

In C++ (especially in OS code where structures can be found at fixed addresses), it can be useful to construct an object in memory that is obtained elsewhere. This is accomplished through a technique known as 'placement new'. For example, say you wanted to create an APIC object at address 0x09FFF0000, then this snippet of code will use placement new to do the trick:

```
void *apic_address = reinterpret_cast<void *>(0x09FFF0000);
APIC *apic = new (apic_address) APIC;
```

Device driver buffers:

- Device drivers might pre-allocate buffers for communication with hardware. Placement new could be used to construct data structures (like headers) within these buffers to manage the data.
- Without placement new, N device driver buffers may be placed at different locations in memory
- Harder to debug, harder to visualize.



Process blocks:

- This is useful for efficiently dividing process memory into its defined components: stack, heap, symbol table, etc.

Fixed memory address range for user processes:

- We can define a fixed memory block where N processes get stored – the user processs space vs the OS space.

Purpose of “placement new” in the flat memory model:

- In our previous example, since we’re tracking memory blocks in data structures like vectors and accessing them via unordered map, “placement new” could be determined for identifying the “starting points” for each block, or a “single entry point” for the whole memory block.
- “Placement new” could be used for more granular tracking of memory addresses → becomes more manual → pointer to available memory, plus using sizeof() for allocation.

Activity:

- Show a working application emulating the first-fit memory allocator for a main memory with $2^{14} = 16,384$ bytes of memory.
 - Set the following configuration for your latest OS emulator:
 - num-cpu 2
 - scheduler “rr”
 - quantum-cycles 4
 - preemptive 1
 - batch-process-freq 0.25
 - min-ins 100
 - max-ins 100
 - Set the newest parameters for your OS emulator memory manager:
 - max-overall-mem 16384
 - min-mem-per-proc 16
 - max-mem-per-proc 4096

- Each process will have a random amount of memory required: $2^4(16) \leq M \leq 2^{12}(4096)$. Note: This is your “min-mem-per-proc” and “max-mem-per-proc” in your MCO2 specs.
- Each process stays in the memory during its execution. It will be released from memory every time it is pre-empted/finished execution.
- If memory is full when a process is scheduled, it will revert back to the tail of the ready queue.
- For every quantum-cycles, produce a text file with the following information:
 - Timestamp
 - Number of processes in memory
 - Total external fragmentation in KB
 - An ASCII printout of the memory.
 - See mockup below:

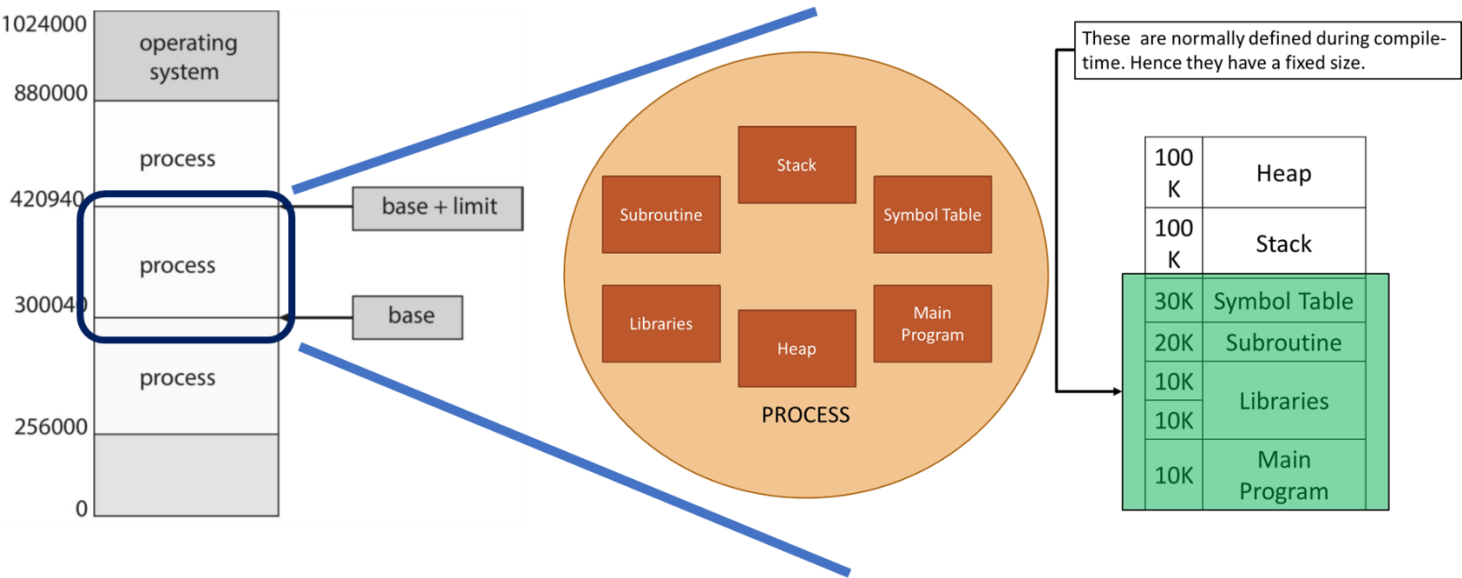

```

168      Timestamp: (08/06/2024 09:15:22AM)
169      Number of processes in memory: 2
170      Total external fragmentation in KB: 8192
171
172      ----end---- = 16384
173
174      16384
175      P1
176      12288
177
178      8192
179      P9
180      4096
181
182      ----start---- = 0
          
```
 - Explanation: The boundaries indicate the memory address limits. For every process, print its upper limit, and lower limit, following the prescribed layout. In this example, there are only 2 processes in memory. Then, there’s external fragmentation of: $12,288(P1\ lower\ bound) - 8,192(P2\ upper\ bound) + 4096(P2\ lower\ bound) - 0(start\ of\ address) = 8,192\ bytes$.

Paging

- Most OS paging schemes.
- This method allows non-contiguous allocation of memory for processes → although internal fragmentation may still occur.

Recall that a process is stored in memory and divided into subspaces.



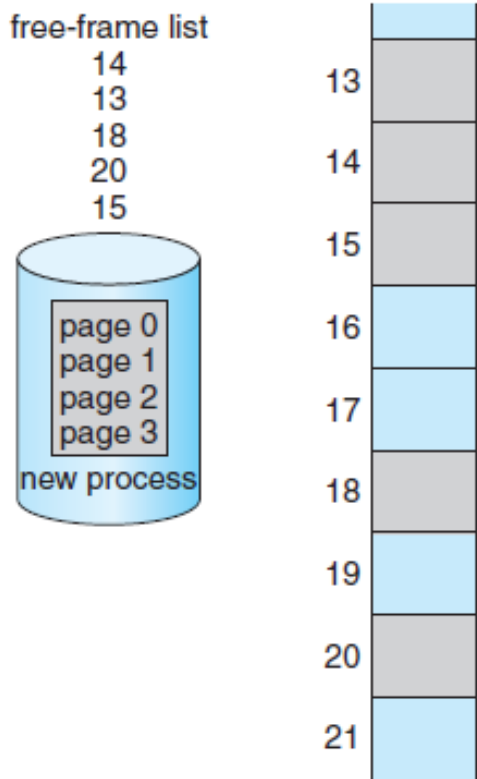
Paging Concepts

The subspaces in each processes are referred to as **pages**.

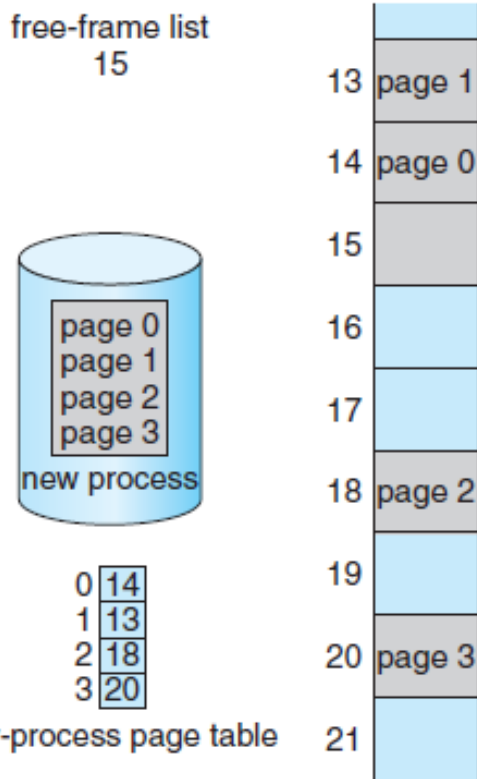
100 K	Heap	0
100 K	Stack	1
30K	Symbol Table	2
20K	Subroutine	3
10K	Libraries	4
10K		
10K	Main Program	5

1. The OS maintains a list of free frames.
2. All processes that arrive have a set number of pages.
3. The OS selects free frame lists to assign the pages and creates the page table.
4. Processes that finishes execution returns the free frames.

This is illustrated in the following diagram:



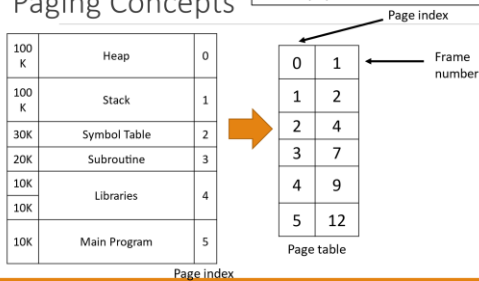
(a)



(b)

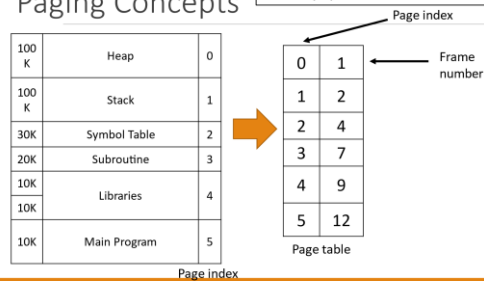
Paging Concepts

A page table is maintained by the OS which determines the EXACT frame number in the physical memory space.

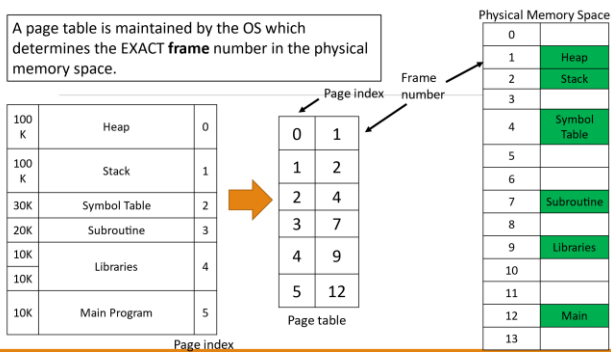


Paging Concepts

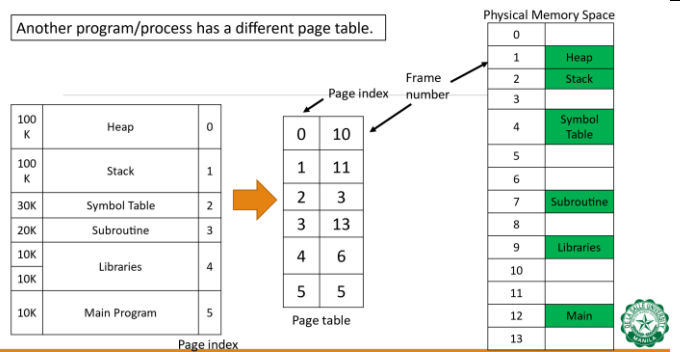
A page table is maintained by the OS which determines the EXACT frame number in the physical memory space.



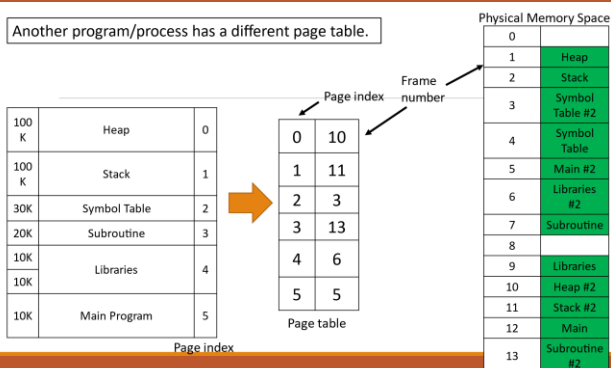
A page table is maintained by the OS which determines the EXACT frame number in the physical memory space.



Another program/process has a different page table.



Another program/process has a different page table.



Assumptions:

- All pages must be loaded in memory for the program to run.
- All pages must be unloaded after the program finishes.

Paging Problem #1

5. Consider processes wherein each need **5 pages** to execute. The OS has 15 frames available and the **free-frame** list is treated as a queue and has the following values: 13,14,15,4,5,6,7,8,0,1,2,3,9,10,12,11. If the **free-frame** list is empty, the process will need to wait. Consider the process table and using a CPU scheduling scheme of **pre-emptive shortest-job first**, answer the following questions.

Process	Arrival Time	CPU Burst
P1	0	10
P2	1	8
P3	2	8
P4	3	3
P5	4	3

- A. Draw the **physical memory map** by time slice 10.
- B. What will be the last value of the **free-frame** list after all processes have finished executing?
- C. Indicate **all free frames** by time slice 10.
- D. Draw the **physical memory map** by time slice 15.

Paging Problem #2

6. Consider processes wherein each need **5 pages** to execute. The OS has 10 frames available and the **free-frame** list is treated as a queue and has the following values: 4,5,6,7,0,1,2,3,8,9. The OS has **swapping enabled** and performs swapping on the oldest process. Consider the process table and using a CPU scheduling scheme of **pre-emptive shortest job first**, answer the following questions.

Process	Arrival Time	CPU Burst
P1	0	10
P2	5	5
P3	10	13
P4	12	10
P5	14	5

- A. What will be the last value of the **free-frame** list after all processes have finished executing?
- B. Indicate the allocated **frames** for P1.
- C. Indicate the allocated **frames** for P2.
- D. Indicate the allocated **frames** for P3.
- E. Indicate the allocated **frames** for P4.
- F. Indicate the allocated **frames** for P5.

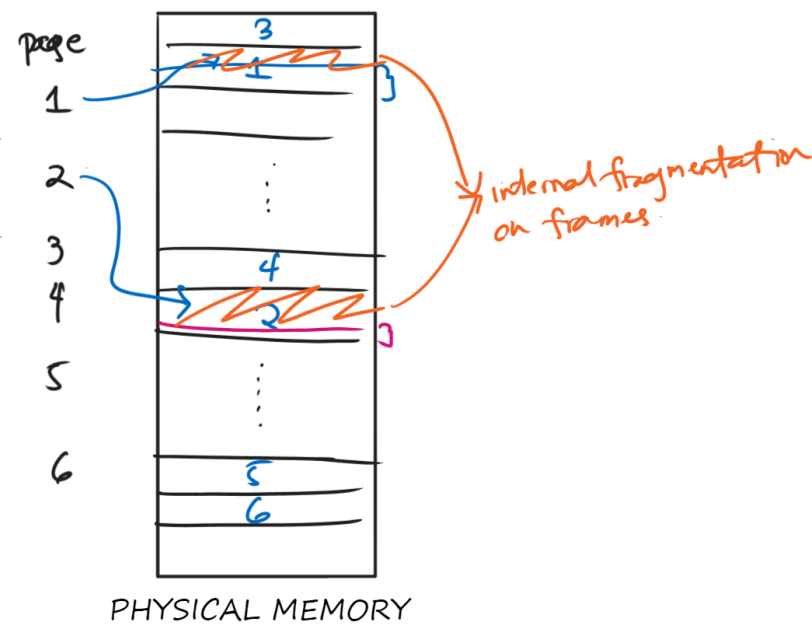
Internal fragmentation only happens within the pages themselves but they are very minimal → heap and stack pre-allocations.

Consider a running process

100K	Heap
100K	Stack
30K	Symbol Table
20K	Subroutine
10K	Libraries
10K	Main Program

← currently used space

← currently used space



Emulating paging and backing store

Code translations:

- The process, memory allocator, and scheduling classes discussed previously can be extended to support paging.
 - Paging → a process must have defined page numbers. Following the components of a program, we could design each process to have **six** pages with the following indices:
- | Page index | Purpose |
|------------|--------------|
| 0 | Heap |
| 1 | Stack |
| 2 | Symbol table |
| 3 | Subroutine |
| 4 | Libraries |
| 5 | Main program |
- For simplicity, we could limit each page to a fixed sized → placeholder sizeof(char) or sizeof(int). We are merely simulating the memory usage of each page.
 - A class must hold the free frame list and how each frame is mapped to a memory block → PagingAllocator
 - The paging allocator must compute the ideal number of frames to be allocated to a given memory size → the physical memory block.

Sample code template:

```
265 class IMemoryAllocator {
266 public:
267     virtual void* allocate(Process* process) = 0;
268     virtual void deallocate(Process* process) = 0;
269     virtual void visualizeMemory() const = 0;
270 };
271
272 class PagingAllocator : public IMemoryAllocator {
273 public:
274     PagingAllocator(size_t maxMemorySize);
275
276     void* allocate(Process* process) override;
277     void deallocate(Process* process) override;
278     void visualizeMemory() const override;
279
280 private:
281     size_t maxMemorySize;
282     size_t numFrames;
283     std::unordered_map<size_t, size_t> frameMap;
284     std::vector<size_t> freeFrameList;
285
286     size_t allocateFrames(size_t numFrames, size_t processId, const std::vector<size_t>& pageSizes);
287     void deallocateFrames(size_t numFrames, size_t frameIndex, const std::vector<size_t>& pageSizes);
288 };
```

The paging allocator implementation

```

324 // Implementation of PagingAllocator class
325 PagingAllocator::PagingAllocator(size_t maxMemorySize)
326 : maxMemorySize(maxMemorySize), numFrames(maxMemorySize) {
327     // Initialize the free frame list
328     for (size_t i = 0; i < numFrames; ++i) {
329         freeFrameList.push_back(i);
330     }
331 }
332
333 void* PagingAllocator::allocate(Process* process) {
334     size_t processId = process->getId();
335     size_t numFramesNeeded = process->getNumPages();
336     if (numFramesNeeded > freeFrameList.size()) {
337         std::cerr << "Memory allocation failed. Not enough free frames.\n";
338         return nullptr;
339     }
340
341     // Allocate frames for the process
342     size_t frameIndex = allocateFrames(numFramesNeeded, processId);
343     return reinterpret_cast<void*>(frameIndex);
344 }
345
346 void PagingAllocator::deallocate(Process* process) {
347     size_t processId = process->getId();
348
349     // Find frames allocated to the process and deallocate
350     auto it = std::find_if(frameMap.begin(), frameMap.end(),
351         [processId](const auto& entry) { return entry.second == processId; });
352
353     while (it != frameMap.end()) {
354         size_t frameIndex = it->first;
355         deallocateFrames(1, frameIndex);
356         it = std::find_if(frameMap.begin(), frameMap.end(),
357             [processId](const auto& entry) { return entry.second == processId; });
358     }
359 }
360
361 void PagingAllocator::visualizeMemory() const {
362     std::cout << "Memory Visualization:\n";
363     for (size_t frameIndex = 0; frameIndex < numFrames; ++frameIndex) {
364         auto it = frameMap.find(frameIndex);
365         if (it != frameMap.end()) {
366             std::cout << "Frame " << frameIndex << " -> Process " << it->second << "\n";
367         } else {
368             std::cout << "Frame " << frameIndex << " -> Free\n";
369         }
370     }
371     std::cout << "-----\n";
372 }
373
374 size_t PagingAllocator::allocateFrames(size_t numFrames, size_t processId, const std::vector<size_t>& pageSizes) {
375     size_t frameIndex = freeFrameList.back();
376     freeFrameList.pop_back();
377
378     // Map allocated frames to the process ID
379     for (size_t i = 0; i < numFrames; ++i) {
380         frameMap[frameIndex + i] = processId;
381     }
382
383     return frameIndex;
384 }
385
386 void PagingAllocator::deallocateFrames(size_t numFrames, size_t frameIndex, const std::vector<size_t>& pageSizes) {
387     // Remove mapping of deallocated frames
388     for (size_t i = 0; i < numFrames; ++i) {
389         frameMap.erase(frameIndex + i);
390     }
391
392     // Add frames back to the free frame list
393     for (size_t i = 0; i < numFrames; ++i) {
394         freeFrameList.push_back(frameIndex + i);
395     }
396 }

```

Consider a sample paging scheduler (FCFS scheduler) below that uses the paging allocator:

```

407 void PagingScheduler::addProcess(Process* process) {
408     // Allocate memory for the process
409     void* memoryBlock = memoryAllocator.allocate(process);
410     if (!memoryBlock) {
411         std::cerr << "Failed to add process " << process->getId() << ". Memory allocation failed.\n";
412         return;
413     }
414
415     size_t coreId = rand() % numCores; // Assign the process to a random core
416     processCores[process] = coreId;
417     process->setMemoryBlock(memoryBlock);
418     std::cout << "Process " << process->getId() << " added to Core " << coreId << ".\n";
419 }
420
421 void PagingScheduler::removeProcess(Process* process) {
422     // Deallocate memory for the process
423     memoryAllocator.deallocate(process);
424     processCores.erase(process);
425     std::cout << "Process " << process->getId() << " removed.\n";
426 }
427
428 void PagingScheduler::visualizeMemory() const {
429     memoryAllocator.visualizeMemory();
430 }
431
432 void PagingScheduler::simulateExecution() {
433     for (auto& thread : coreThreads) {
434         thread.join();
435     }
436 }
437
438 void PagingScheduler::coreFunction(size_t coreId) {
439     <do some CPU work>
440 }

```

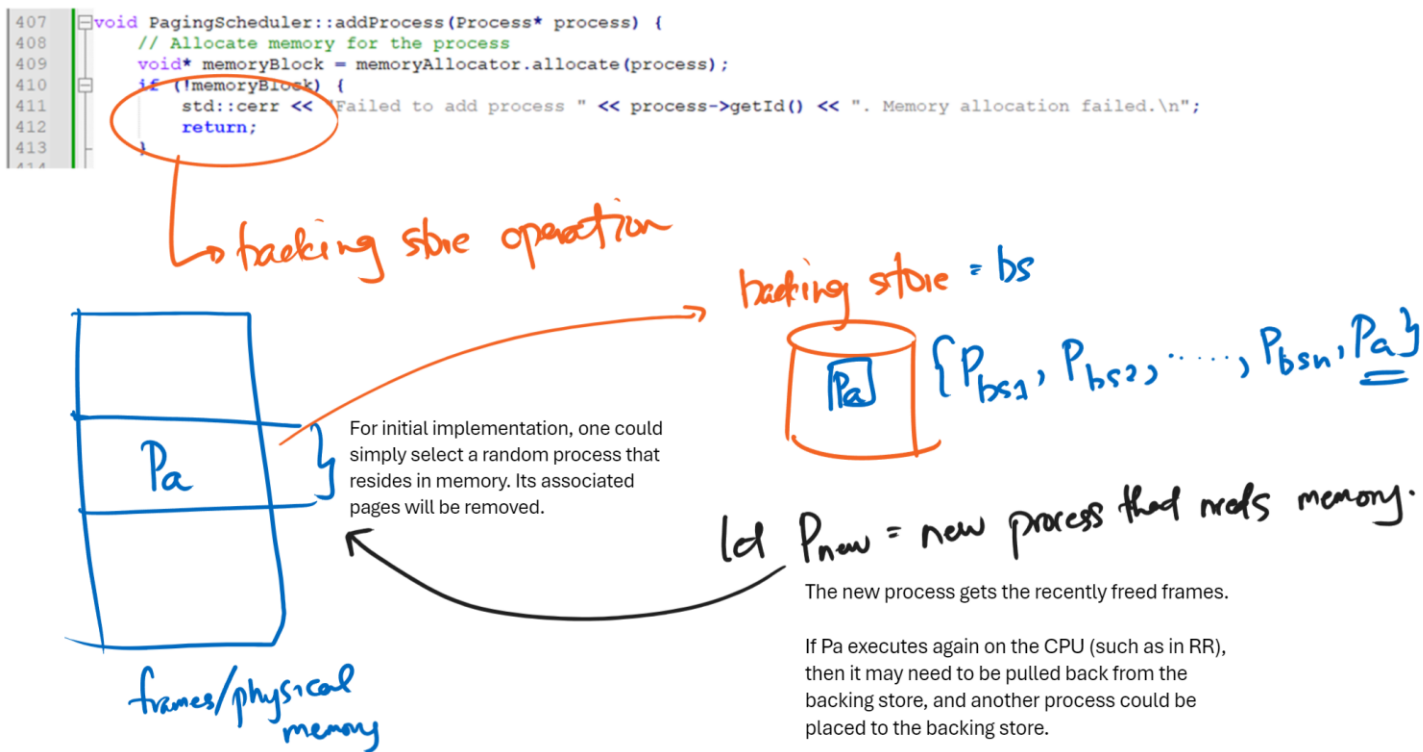
Sample usage:

```

442 int main() {
443     // Create processes with specified page sizes
444     Process process1(1, {256, 256, 256, 256, 256});
445     Process process2(2, {256, 256, 256, 256, 256});
446     Process process3(3, {256, 256, 256, 256, 256});
447     Process process4(4, {256, 256, 256, 256, 256});
448     Process process5(5, {256, 256, 256, 256, 256});
449
450     // Set up paging scheduler with 4 cores and maximum memory size
451     PagingScheduler scheduler(4, 4096);
452
453     // Add processes to the scheduler
454     scheduler.addProcess(&process1);
455     scheduler.addProcess(&process2);
456     scheduler.addProcess(&process3);
457     scheduler.addProcess(&process4);
458     scheduler.addProcess(&process5);
459
460     // Visualize initial memory state
461     scheduler.visualizeMemory();
462
463     // Simulate process execution
464     scheduler.simulateExecution();
465
466     // Processes automatically deallocated upon destruction
467
468     return 0;
469 }

```

Backing store operation



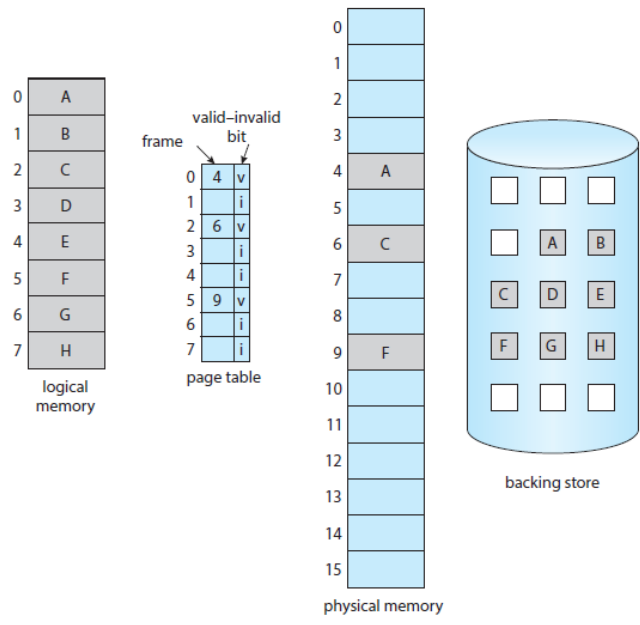
- To emulate a backing store:
- A backing store could be text files saved in a certain directory. E.g. a backing store directory relative to your CSOPESY emulator path.
 - Why must we save to text files? We must free our **main memory** usage.
 - What should we save in our text files? All necessary process info must be saved, such as:
 - Process id, name, command counter
 - The number of pages and associated memory sizes, as dictated by its last command execution.In our case, since process memory and page size is fixed, we simply store these values.

Demand paging

- To further conserve and have a more efficient memory use, we must ensure that the process/es selected for CPU execution already resides in our main memory.
- To ensure that we have free frames as much as possible, an alternative strategy is to load pages only as they are needed – demand paging.

Basic concepts

- The page table is extended to indicate a valid/invalid bit. If the page index is marked as valid, it means it is in main memory. If it is invalid, it resides in the backing store.
- At the start of execution, an active process will have **all its pages** in the backing store and marked as invalid.
- At any point in execution, an active process can have **some of its pages** put to main memory, and some put back into a backing store if it's not needed → another active process can then load its needed pages into main memory.



Demand paging

100 K	Heap	P₁₅ P ₁₄ P ₁₃ P ₁₂ P ₁₁ P ₁₀ P ₉ P ₈ P ₇ P ₆ P ₅ P ₄ P ₃ P ₂ P ₁
100 K	Stack	
30K	Symbol Table	
20K	Subroutine	
10K	Libraries	
10K		
10K	Main Program	

We can fine-grain the required pages per process, where subcomponents may require multiple pages.

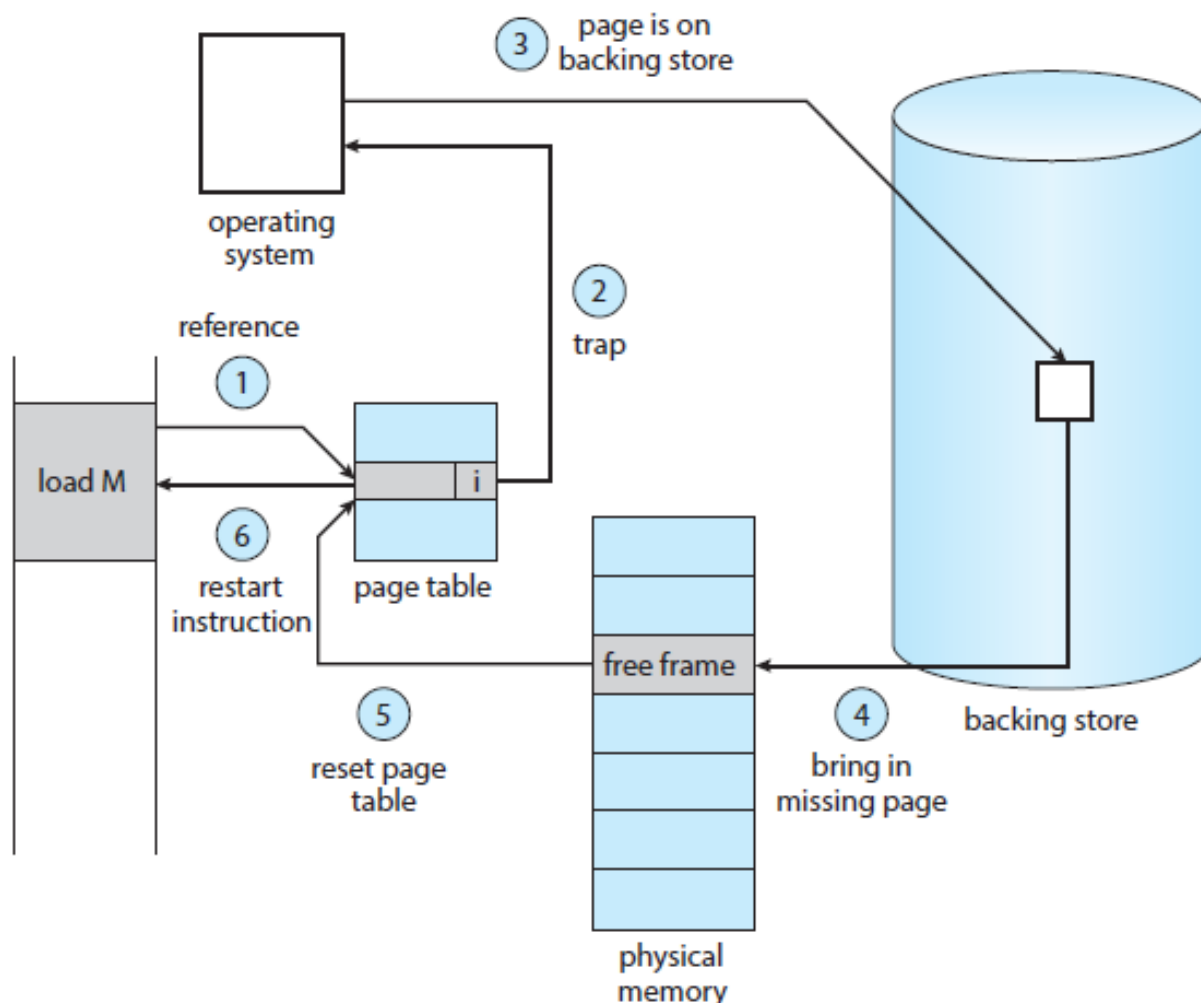
Purpose: Allows processes with smaller memory footprint to run with other processes that need larger memory.

The compiler and/or OS typically decides for the page allocation of each process.

Paging instructions and page fault

- If some pages are in a backing store, then this implies that a process may inevitably access a page that is not yet in main memory.
- Such incident is called a **page fault** – access to a page marked invalid.

The procedure for handling page faults are as follows (see illustration) :



1. We check an internal table (page table) for this process to determine whether the reference was a valid or invalid memory access.
2. If the reference is invalid, we terminate the process (access violation error. This happens in system-level programming languages where memory allocation is explicit). If it was valid but we have not yet brought in that page, we now page it in.
3. We find a free frame.
4. We schedule a secondary storage operation to read the desired page into the newly allocated frame.
5. When the storage read is complete, we modify the internal table kept with the process and the page table to indicate that the page is now in memory.
6. We restart the instruction that was interrupted. The process can now access the page as though it had always been in memory.

Theoretically, some programs could access several new pages of memory with each instruction execution (one page for the instruction and many for data), possibly causing multiple page faults per instruction. This situation would result in unacceptable system performance. Fortunately, analysis of running processes shows that this behavior is exceedingly unlikely. Programs tend to have locality of reference which results in reasonable performance from demand paging.

The hardware to support demand paging is the same as the hardware for paging and swapping:

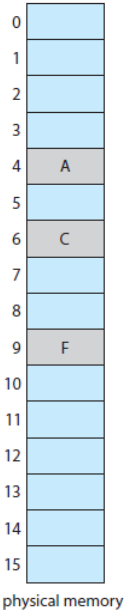
- **Page table.** This table has the ability to mark an entry invalid through a valid-invalid bit or a special value of protection bits.
- **Secondary memory.** This memory holds those pages that are not present in the main memory. The secondary memory is usually a high-speed disk or NVM device. It is known as the swap device, and the section of storage used for this purpose is known as swap space/backing store.

Maintaining free frames

- Most OS maintain a free-frame list, a pool of free frames for satisfying such requests.
- Free frames are typically allocated when the stack or heap segments from a process expand.



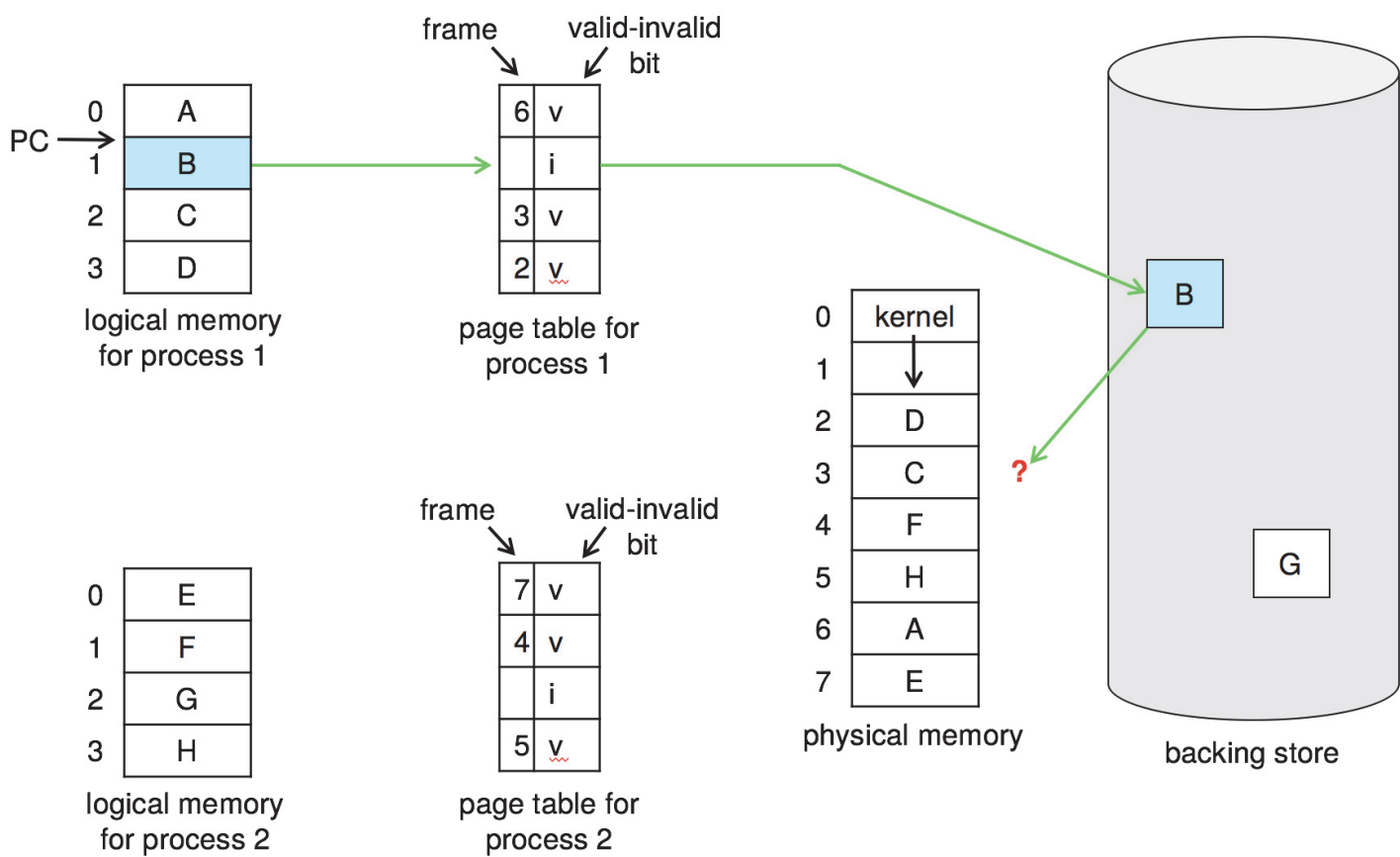
- When a system starts up, all available memory is placed on the free-frame list. As free frames are requested (for example, through demand paging), the size of the free-frame list shrinks. At some point, the list either falls to zero or falls below a certain threshold, at which point it must be repopulated – page replacement.



Page replacement

- As OS tries to run multiple processes, the degree of multi-programming increases → high risk and frequency of running out of free frames.
- The operating system has several options at this point:
 - Terminate the process
 - Easy to implement, but not good for the user.
 - Demand paging is the operating system’s attempt to improve the computer system’s utilization and throughput.
 - Users should not be aware that their processes are running on a paged system—paging should be logically transparent to the user. So, this option is not the best choice.
 - Perform page replacement → if no frame is free, then find a candidate page being used and free it – but which page?

Page replacement is illustrated below:



The general algorithm for **page replacement** is as follows:

1. Find the location of the desired page on secondary storage.
2. Find a free frame:
 - a. If there is a free frame, use it.
 - b. If there is no free frame, use a page-replacement algorithm to select a victim frame.
 - c. Write the victim frame to secondary storage; change the page and frame tables accordingly.
3. Read the desired page into the newly freed frame; change the page and frame tables.
4. Continue the process from where the page fault occurred.

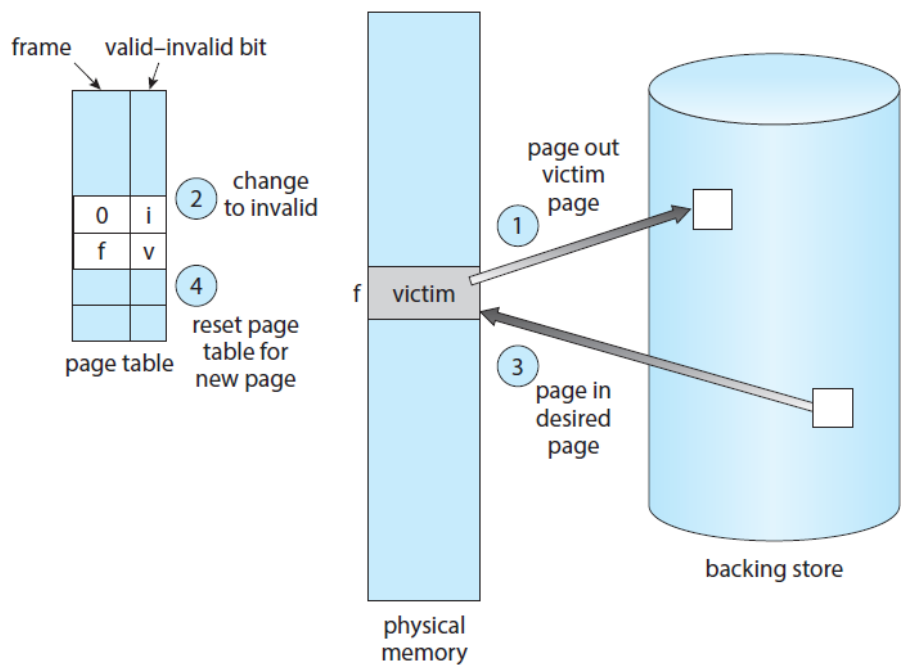


Figure 10.10 Page replacement.

- With page replacement, we technically have infinite virtual memory
- Memory seen by the OS and processes isn't directly tied to the limits of the physical memory.
 - Assume that there are no free frames, OS will always find a victim frame to replace

There are different page replacement algorithms:

Random:

- **Principle:** Randomly selects a page to evict.
- **Pros:** Simplicity.
- **Cons:** Unpredictable and may not perform well in practice.

FIFO (First-In-First-Out):

- **Principle:** Evict the oldest page in the memory queue.
- **Pros:** Simple and easy to implement.
- **Cons:** May suffer from the "Belady's Anomaly," where increasing the number of frames does not guarantee a reduction in page faults.

Optimal Page Replacement:

- **Principle:** Evict the page that will not be used for the longest period (optimal, but not practically implementable).
- **Pros:** Theoretical optimal solution.
- **Cons:** Requires future knowledge of page accesses, which is not feasible in a real system.

LRU (Least Recently Used):

- **Principle:** Evict the page that has not been used for the longest time.
- **Pros:** Intuitive and often effective.
- **Cons:** Implementation complexity. Keeping track of usage timestamps can be resource-intensive.

```

478 // Process class (Assuming each page has a fixed size)
479 class Process {
480 public:
481     Process(int id, std::vector<int> pageIndices) : id(id), pageIndices(pageIndices) {}
482
483     int getId() const { return id; }
484     const std::vector<int>& getPageIndices() const { return pageIndices; }
485
486 private:
487     int id;
488     std::vector<int> pageIndices;
489 };
490
491 // FIFO Page Replacement Algorithm
492 class FIFOPageReplacement {
493 public:
494     FIFOPageReplacement(int frameCount) : frameCount(frameCount) {}
495
496     void loadProcess(const Process& process) {
497         std::cout << "Loading Process " << process.getId() << " into memory...\n";
498
499         for (int pageIndex : process.getPageIndices()) {
500             if (pageTable.find(pageIndex) == pageTable.end()) {
501                 // Page fault: Page is not in memory
502                 if (memoryQueue.size() >= frameCount) {
503                     // Evict the oldest page if memory is full
504                     int oldestPage = memoryQueue.front();
505                     memoryQueue.pop();
506                     pageTable.erase(oldestPage);
507                     std::cout << "Evicting Page " << oldestPage << "\n";
508                 }
509
510                 // Load the new page into memory
511                 memoryQueue.push(pageIndex);
512                 pageTable.insert(pageIndex);
513                 std::cout << "Page " << pageIndex << " loaded into memory.\n";
514             } else {
515                 // Page is already in memory
516                 std::cout << "Page " << pageIndex << " is already in memory.\n";
517             }
518         }
519
520         std::cout << "Process " << process.getId() << " execution completed.\n\n";
521     }
522
523 private:
524     int frameCount;
525     std::queue<int> memoryQueue;
526     std::unordered_set<int> pageTable;
527 };
528
529 int main() {
530     // Create a FIFO page replacement object with 3 frames
531     FIFOPageReplacement fifoPageReplacement(3);
532
533     // Create processes with page indices
534     Process process1(1, {0, 1, 2});
535     Process process2(2, {3, 4, 5});
536     Process process3(3, {0, 1, 2});
537     Process process4(4, {6, 7, 8});
538
539     // Load processes into memory using FIFO page replacement
540     fifoPageReplacement.loadProcess(process1);
541     fifoPageReplacement.loadProcess(process2);
542     fifoPageReplacement.loadProcess(process3);
543     fifoPageReplacement.loadProcess(process4);
544
545     return 0;
546 }

```

Thrashing

- Consider what occurs if a process does not have “enough” frames—that is, it does not have the minimum number of frames it needs to support pages in the working set. The process will quickly page-fault.
- At this point, it must replace some page. However, since all its pages are in active use, it must replace a page that will be needed again right away.
- Consequently, it quickly faults again, and again, and again, replacing pages that it must bring back in immediately.
- This high paging activity is called thrashing. A process is thrashing if it is spending more time paging than executing.
- This incident typically occurs when there are already so many processes running in a system.

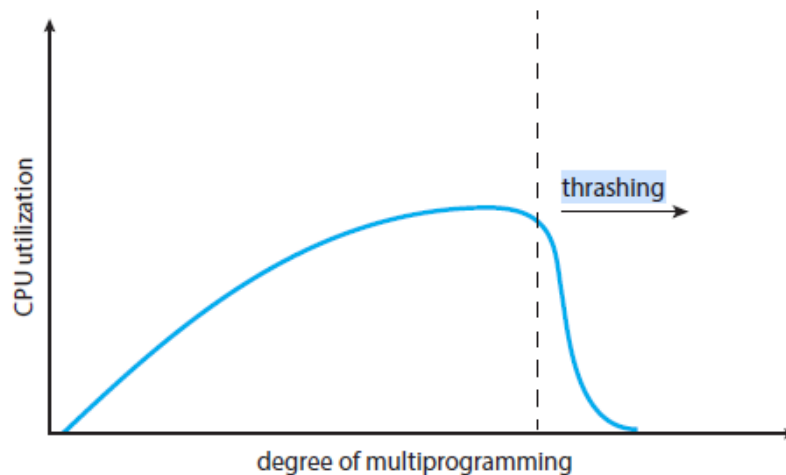


Figure 10.20 Thrashing.

Difference between paging and demand paging

- **Paging:**
 - Divides memory into fixed-sized blocks called pages.
 - When a process executes, it splits into pages.
 - When the process requests memory, the operating system allocates page frames from the primary memory.
 - The operating system moves program pages from the secondary memory (SSD/HDD) to the primary memory frames.
 - This technique improves the efficiency of memory management by eliminating external fragmentation.
- **Demand Paging:**
 - It's a specific implementation of paging.
 - The operating system loads only the necessary pages of a program into memory at runtime.
 - A page fault occurs when the program needs to access a page that is not currently in memory.
 - The operating system then loads the required pages from the disk into memory.
 - The operating system updates the page tables accordingly.
 - This process is transparent to the running program.

Memory-mapped files and shared libraries

- To further reduce the need for page replacement and maximize the efficient use of main memory – we can dedicate portions of the main memory for **shared data**.
- Summary: Processes should only have their dedicated memory space used for unique data. Any data that could be potentially shared must reside only once in the main memory.

Shared libraries:

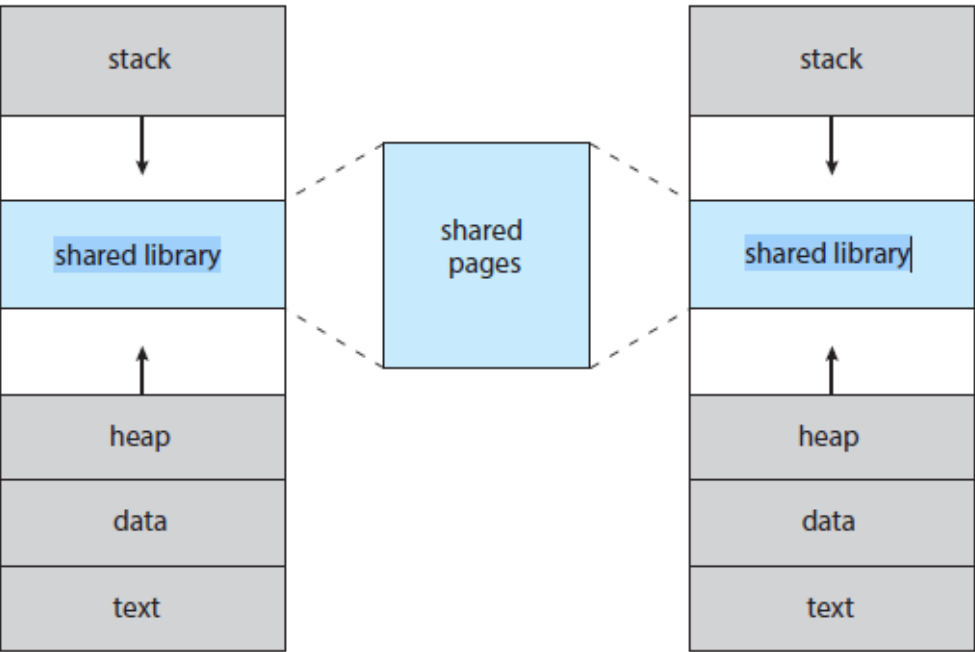
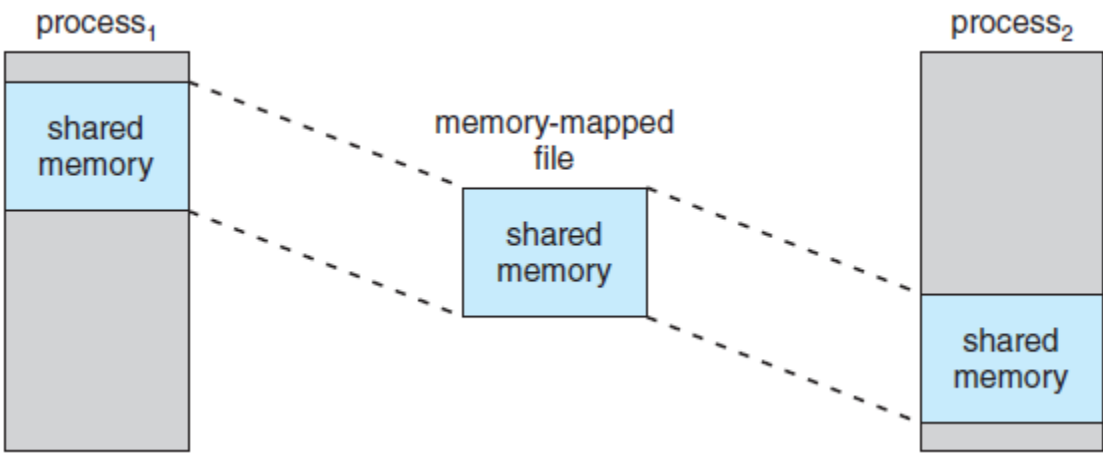


Figure 10.3 Shared library using virtual memory.

- System libraries such as the standard C library can be shared by several processes through mapping of the shared object into a virtual address space.
- Although each process considers the libraries to be part of its virtual address space, the actual pages where the libraries reside in physical memory are shared by all the processes (Figure 10.3). Typically, a library is mapped read-only into the space of each process that is linked with it.

Memory-mapped files:

- Recall that each process will have some pre-allocated memory.
 - Typically the heap/stack is pre-allocated and can run out of space, especially for operations that perform reading/writing of big datasets.
- Memory-mapped files are a mechanism that enables a file or a portion of a file to be directly mapped into the virtual memory of a process. This allows the process to access the file using memory references rather than traditional read and write operations.
- Memory-mapped files are commonly used in scenarios where a large dataset needs to be accessed in a way that is more efficient than traditional file I/O, such as database systems, certain types of file processing, and shared-memory IPC (Inter-Process Communication).



All programming languages provide support for memory-mapped files to allow developers to code applications that would require large files to be read.

